

When Agda met Vampire

Artjoms Šinkarovs and Michael Rawson

Jia-Wei Zhang

08/2026

Motivation

Background

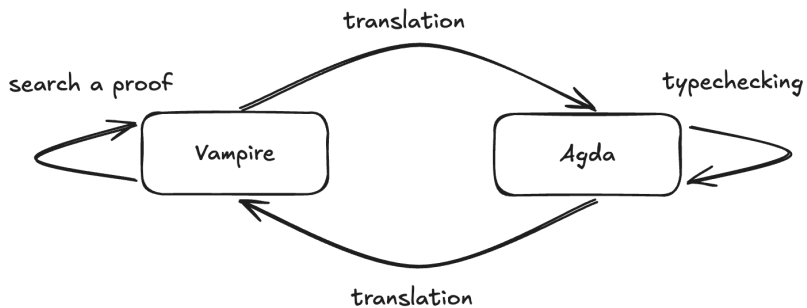
ATP	ITP
Automated Theorem Proving ▶ Machine searches for proof automatically ▶ ATPs are based on Classical FOL ▶ Modern ATPs: E, Vampire	Interactive Theorem Proving ▶ Human guides proof; machine checks every step ▶ Most ITPs are based on Intuitionistic Logic ▶ Proof Assistants: Agda, Rocq, Lean

Vampire, as the one of state-of-the-art ATP, has been under development for nearly 35 years.

Since 1999, Vampire won 78 titles in CASC.

Vampire won all competitions in 2025 and 2026!!

Motivation



Example 1: Vector Spaces (without Scalars)

```
postulate
  V      : Set
  ze     : V
  _+_    : V -> V -> V
  _-     : V -> V
  -- (declare-sort V 0)
  -- (declare-const ze V)
  -- (declare-fun _+_ (V V) V)
  -- (declare-fun _- (V) V)

  neut1  : forall u -> ze + u = u
  -- (assert (! (forall ((u V))
  --   (= (_+_ ze u) u)) :named neut1))
  neg1   : forall u -> (- u) + u = ze
  -- (assert (! (forall ((u V))
  --   (= (_+_ (- u) u) ze))
  --   :named neg1))
  assoc  : forall u v w ->
    (u + v) + w = u + (v + w)
  -- (assert (! (forall ((u V) (v V) (w V))
  --   (= (_+_ (_+_ u v) w)
  --     (_+_ u (_+_ v w)))) :named assoc))
  -- (declare-const u V)
  -- (declare-const v V)
  ze-uniq' : forall u v ->
    u + v = v -> u = ze
  -- (assert (! (forall ()
  --   (= (_+_ u v) v)) :named e))
  -- (assert-not (! (= u ze) :named ze-uniq))
```

Agda to Vampire is based on **Agda reflection**.
Agda reflection is built-in metaprogramming system.

A Little Taste of Agda

```
trivial : forall u -> -u + (u + u) = u

t1 : -u + (u + u) = (-u + u) + u
t1 = sym (assoc (-u) u u)

t2 : (-u + u) + u = ze + u
t2 = cong (_+ u) (negl u)

ans : -u + (u + u) = u
ans = trans t1 t2
```

```
sym : {A : Set} {x y : A} x ≡ y → y ≡ x
cong : {A B : Set} {x y : A} (f : A → B) → x ≡ y → f x ≡ f y
trans : {A : Set} {x y z : A} x ≡ y → y ≡ z → x ≡ z
negl : {V : Set} {u : V} (- u) + u = ze
assoc : {V : Set} {u v w : V} (u + v) + w = u + (v + w)
```

Contents

Motivation

Classical FOL

Vampire Core Calculus

Vampire to Agda

Thesis Proposal

References

Classical FOL

Classical FOL

$x\ y\ z$ as individual variable.

$$term ::= x \mid c \mid f(term, \dots, term)$$
$$atom ::= P(term, \dots, term) \mid term = term \mid \top \mid \perp$$
$$literal ::= atom \mid \neg atom$$
$$\begin{aligned} \varphi ::= & literal \mid \neg \varphi \mid \varphi \wedge \varphi \\ & \mid \varphi \rightarrow \varphi \mid \varphi \leftrightarrow \varphi \\ & \mid \forall x \varphi \mid \exists x \varphi \end{aligned}$$

The key point of translation: $P \rightarrow Q \equiv \neg P \vee Q$

Vampire is based on **refutation proof**.

(It is likely **proof by contradiction**.)

Horn Clause and Notation

A **first-order clause** is a $\forall$ -quantified disjunction of literals.

► **Horn clause:** 0 or 1 positive “head” literals

► $\forall xy \dots L_1 \vee L_2 \vee \dots$

► **definite clause:** exactly 1 positive literal

► $\forall xy \dots \neg A_1 \vee \neg A_2 \vee \dots \vee B$

► $\forall xy \dots A_1 \wedge A_2 \wedge \dots \rightarrow B$

► **goal clause:** exactly 0 positive literals

► $\forall xy \dots \neg A_1 \vee \neg A_2 \vee \dots \vee \neg A_n$

► $\forall xy \dots A_1 \wedge A_2 \wedge \dots \wedge A_n \rightarrow \perp$

each L_i are Literal and each A_i and B are Atom.

equations $s = t$ are treated as implicitly symmetric.

Vampire Core Calculus

Vampire Core Calculus

$$\frac{A \vee C \quad \neg A' \vee D}{\sigma(C \vee D)} \text{ RESOLUTION}$$

$$\frac{A \vee A' \vee C}{\sigma(A \vee C)} \text{ FACTORING}$$

$$\frac{l = r \vee C \quad A[l'] \vee D}{\sigma(A[r] \vee C \vee D)} \text{ SUPERPOSITION} \quad \frac{s \neq t \vee C}{\sigma(C)} \text{ EQ. RESOLUTION}$$

$\text{mgu}(A, A')$ in Resolution and Factoring.

$\text{mgu}(l, l')$ in Superposition.

$\text{mgu}(s, t)$ in Equality Resolution.

$A, A[l']$ are atoms.

C, D are clauses.

l, l', r, s, t are terms.

Example 1 (Vector Spaces)

```
1. forall x. 0 + x = x (neut1)
2. forall x. (-x) + x = 0 (neg1)
3. forall xyz. (x+y)+z = x+(y+z) (assoc)
4. v = u + v (e)
5. 0 != u (negated goal)
6. forall xy. -x+(x+y) = 0+y (superposition 3, 2)
7. forall x. v+x = u+(v+x) (superposition 3, 4)
8. forall xy. -x+(x+y) = y (superposition 6, 1)
9. forall x. (--x)+0 = x (superposition 8, 2)
10. forall xy. y+x = (--y)+x (superposition 8, 8)
11. forall x. x+0 = x (superposition 9, 10)
12. forall x. 0 = x+(-x) (superposition 2, 10)
13. 0 = u+0 (superposition 7, 12)
14. 0 = u (superposition 13, 11)
15. false (resolution 14, 5)

-- Vampire Original Output
1. 'vec._+_('vec.ze',X0) = X0
2. 'vec._+_('vec._-'(X0),X0) = 'vec.ze'
3. 'vec._+_('vec._+_'(X0,X1),X2) = 'vec._+_'(X0,'vec._+_'(X1,X2))
4. v = 'vec._+_'(u,v)
5. 'vec.ze' != u
6. 'vec._+_('vec._-'(X0),'vec._+_'(X0,X1)) = 'vec._+_('vec.ze',X1)
7. 'vec._+_'(v,X0) = 'vec._+_'(u,'vec._+_'(v,X0))
8. 'vec._+_('vec._-'(X0),'vec._+_'(X0,X1)) = X1
9. 'vec._+_('vec._-'('vec._-'(X0)), 'vec.ze') = X0
10. 'vec._+_'(X1,X0) = 'vec._+_('vec._-'('vec._-'(X1)),X0)
11. 'vec._+_'(X0,'vec.ze') = X0
12. 'vec.ze' = 'vec._+_'(X0,'vec._-'(X0))
13. 'vec.ze' = 'vec._+_'(u,'vec.ze')
14. 'vec.ze' = u
15. $false
```

Example 1 (Vector Spaces)

$$2 \quad \forall x. (-x) + x = 0 \quad (l = r)$$

$$3 \quad \forall x y z. (x + y) + z = x + (y + z) \quad (A[l'])$$

Sup 3, 2

$$\frac{l = r \quad A[l']}{\sigma(A[r])} \text{ SUP}$$

$$\sigma = \{x \mapsto -x, y \mapsto x\}$$

$$\frac{(-x) + x = 0 \quad ((-x) + x) + z = (-x) + (x + z)}{0 + z = (-x) + (x + z)} \text{ SUP}$$

Vampire to Agda

A classical proof to intuitionistic proof

Restricting both systems to a common language (Horn Clause).
They wish to show $\Gamma \vdash G$, and Vampire obtain a classical proof
by refutation showing $\Gamma, \neg G \vdash \perp$

- ▶ $\Gamma, \neg G \vdash \perp$
- ▶ $\Gamma, G \rightarrow \perp \vdash \perp$
- ▶ $\Gamma, G \rightarrow G \vdash G$ (replacing occurrences of $\perp$ with G)

Γ is “a set of definite clauses”, G is “ground goal atom”.
In general $\perp$ may be woven through the entire proof,
replacing $\perp$ from top right of tree.

Lemmas

- ▶ Lemma 1: If Premises are Horn clause, then Conclusion is also Horn.
- ▶ Lemma 2: Horn Vampire Core Calculus(HVCC) is intuitionistically valid.
- ▶ Lemma 3 (Redundancy Property) : If we finally derived $\perp$ from Horn Vampire Core Calculus(HVCC), then ???

Vampire Core Calculus = VCC, Horn VCC = HVCC

Vampire Core Calculus (General and Horn Clause)

VCC

$$\frac{A \vee C \quad \neg A' \vee D}{\sigma(C \vee D)} \text{ RES.}$$

$$\frac{A \vee A' \vee C}{\sigma(A \vee C)} \text{ FAC.}$$

$$\frac{s \neq t \vee C}{\sigma(C)} \text{ EQ.RES.}$$

$$\frac{l = r \vee C \quad A[l'] \vee D}{\sigma(A[r] \vee C \vee D)} \text{ SUP.}$$

HVCC

$$\frac{\neg P \vee A \quad \neg A' \vee \neg Q \vee \boxed{B}}{\sigma(\neg P \vee \neg Q \vee \boxed{B})} \text{ RES.}$$

$$\frac{\neg A \vee \neg A' \vee \neg P \vee \boxed{B}}{\sigma(\neg A \vee \neg P \vee \boxed{B})} \text{ FAC.}$$

$$\frac{s \neq t \vee \neg P \vee \boxed{A}}{\sigma(\neg P \vee \boxed{A})} \text{ EQ.RES.}$$

$$\frac{\neg P \vee l = r \quad \neg A[l'] \vee \neg Q \vee \boxed{B}}{\sigma(\neg P \vee \neg A[r] \vee \neg Q \vee \boxed{B})} \text{ SUP.L}$$

$$\frac{\neg P \vee l = r \quad \neg Q \vee A[l']}{\sigma(\neg P \vee \neg Q \vee A[r])} \text{ SUP.R}$$

P, Q = premises; A, B = atoms; C, D = clauses;

mgu(A, A'), mgu(l, l'), mgu(s, t)

$\boxed{}$ = positive atom.

$\neg P = \neg(A_1 \wedge A_2 \dots)$, $C = \neg A_1 \vee \neg A_2 \dots$ (Horn and De Morgan)

Vampire Core Calculus (Clause and Implication)

HVCC

$$\frac{\neg P \vee A \quad \neg A' \vee \neg Q \vee \boxed{B}}{\sigma(\neg P \vee \neg Q \vee \boxed{B})} \text{RES.}$$

$$\frac{\neg A \vee \neg A' \vee \neg P \vee \boxed{B}}{\sigma(\neg A \vee \neg P \vee \boxed{B})} \text{FAC.}$$

$$\frac{s \neq t \vee \neg P \vee \boxed{A}}{\sigma(\neg P \vee \boxed{A})} \text{EQ.RES.}$$

$$\frac{\neg P \vee l = r \quad \neg A[l'] \vee \neg Q \vee \boxed{B}}{\sigma(\neg P \vee \neg A[r] \vee \neg Q \vee \boxed{B})} \text{SUP.L}$$

$$\frac{\neg P \vee l = r \quad \neg Q \vee A[l']}{\sigma(\neg P \vee \neg Q \vee A[r])} \text{SUP.R}$$

HVCC^{Imp}

$$\frac{P \rightarrow A \quad A' \wedge Q \rightarrow \boxed{B}}{\sigma(P \wedge Q \rightarrow \boxed{B})} \text{RES.}$$

$$\frac{A \wedge A' \wedge P \rightarrow \boxed{B}}{\sigma(A \wedge P \rightarrow \boxed{B})} \text{FAC.}$$

$$\frac{s = t \wedge P \rightarrow \boxed{A}}{\sigma(P \rightarrow \boxed{A})} \text{EQ.RES.}$$

$$\frac{P \rightarrow l = r \quad A[l'] \wedge Q \rightarrow \boxed{B}}{\sigma(P \wedge A[r] \wedge Q \rightarrow \boxed{B})} \text{SUP.L}$$

$$\frac{P \rightarrow l = r \quad Q \rightarrow A[l']}{\sigma(P \wedge Q \rightarrow A[r])} \text{SUP.R}$$

P, Q = premises; A, B = atoms; $\text{mgu}(A, A'), \text{mgu}(l, l'), \text{mgu}(s, t)$

$\neg P \vee A = P \rightarrow A$

Vampire Core Calculus (Propositions as types)

HVCC^{Imp}

$$\frac{P \rightarrow A \quad A' \wedge Q \rightarrow \boxed{B}}{\sigma(P \wedge Q \rightarrow \boxed{B})} \text{RES.}$$

$$\frac{A \wedge A' \wedge P \rightarrow \boxed{B}}{\sigma(A \wedge P \rightarrow \boxed{B})} \text{FAC.}$$

$$\frac{s = t \wedge P \rightarrow \boxed{A}}{\sigma(P \rightarrow \boxed{A})} \text{EQ.RES.}$$

$$\frac{P \rightarrow l = r \quad A[l'] \wedge Q \rightarrow \boxed{B}}{\sigma(P \wedge A[r] \wedge Q \rightarrow \boxed{B})} \text{SUP.L}$$

$$\frac{P \rightarrow l = r \quad Q \rightarrow A[l']}{\sigma(P \wedge Q \rightarrow A[r])} \text{SUP.R}$$

Agda

$$\frac{t_1 : P \rightarrow A \quad t_2 : A \wedge Q \rightarrow B}{(\lambda(p, q). t_2 (t_1 p, q)) : P \wedge Q \rightarrow B} \text{RES.}$$

$$\frac{t : A \wedge A \wedge P \rightarrow B}{(\lambda(a, p). t (a, a, p)) : A \wedge P \rightarrow B} \text{FAC.}$$

$$\frac{t : s = s \wedge P \rightarrow A}{(\lambda p. t \text{ refl } p) : P \rightarrow A} \text{EQ.RES.}$$

$$\frac{t_1 : P \rightarrow l = r \quad t_2 : A[l] \wedge Q \rightarrow B}{(\lambda(p, a, q). t_2 ((\text{subst } (\text{sym } (t_1 p)) a), q)) : P \wedge A[r] \wedge Q \rightarrow B} \text{SUP.L}$$

$$\frac{t_1 : P \rightarrow l = r \quad t_2 : Q \rightarrow A[l]}{(\lambda p q. \text{subst } (t_1 p) (t_2 q)) : P \wedge Q \rightarrow A[r]} \text{SUP.R}$$

$\text{subst} : l = r \rightarrow A[l] \rightarrow A[r], \text{sym} : l = r \rightarrow r = l.$

Example 2

$$t_1 : \forall x. f(x) = x, \quad t_2 : a = f(c), \quad t_3 : b = c, \quad t_4 : a = b \rightarrow \boxed{\perp} \vdash \boxed{\perp}$$

$$\begin{array}{c}
 \frac{a = f(c) \quad a = b \rightarrow \boxed{\perp}}{\boxed{\perp}} \text{ SUP.L} \\
 \frac{f(c) = c \quad f(c) = b \rightarrow \boxed{\perp}}{\boxed{\perp}} \text{ SUP.L} \\
 \frac{b = c \quad c = b \rightarrow \boxed{\perp}}{\boxed{\perp}} \text{ SUP.L} \\
 \frac{c = c \rightarrow \boxed{\perp}}{\boxed{\perp}} \text{ EQ.RES.}
 \end{array}$$

replace each $\boxed{\perp}$ by $a = b$

Thesis Proposal

Thesis Proposal

- ▶ Formalize $HVCC^{Imp}$ (5 Implication rules) by deep embedding.
- ▶ Prove the $\Gamma, G \rightarrow \perp \vdash \perp$, replace $\perp$ by G .
(Lemma1, 2, 3)

References

- ▶ Artjoms Šinkarovs and Michael Rawson. "When Agda Met Vampire."
- ▶ Laura Kovács and Andrei Voronkov. "Vampire Guide Lectures."
- ▶ Giles Reger. "The Vampire Journey. (Youtube)"
- ▶ Alan J.A. Robinson, Andrei Voronkov. "Paramodulation-Based Theorem Proving"